

EXPERIMENT 7

Aim- Implement the embedding wallet (Metamask) and transaction using Solidity.

Theory-

What is MetaMask?

MetaMask is a cryptocurrency wallet and browser extension that allows users to interact with blockchain networks like Ethereum.

- It stores private keys securely
- Enables sending/receiving crypto (like ETH)
- Allows interaction with decentralized applications (DApps)
- Works as a bridge between your browser and

blockchain What is Testnet?

A test net (test network) is a blockchain environment used for testing smart contracts and transactions without using real money.

- Uses free “test cryptocurrency” (no real value)
- Helps developers debug and experiment safely
- Examples:
 - a. Sepolia Testnet
 - b. Goerli Testnet

List the steps to connect MetaMask with RemixIDE for transactions.

Step 1: Install MetaMask

- Add MetaMask extension to browser
- Create or import a wallet

Step 2: Select a Network

- Open MetaMask
- Choose a test network (e.g., Sepolia)

Step 3: Get Test ETH

- Use a faucet to receive free test ETH

Step 4: Open Remix IDE

- Go to Remix IDE in browser

Step 5: Write/Compile Smart Contract

- Create a Solidity file
- Compile the contract

Step 6: Connect MetaMask to Remix

- Go to Deploy & Run Transactions tab
- In Environment dropdown → select Injected Provider - MetaMask

Step 7: Authorize Connection

- MetaMask will pop up → click Connect

Step 8: Deploy Contract

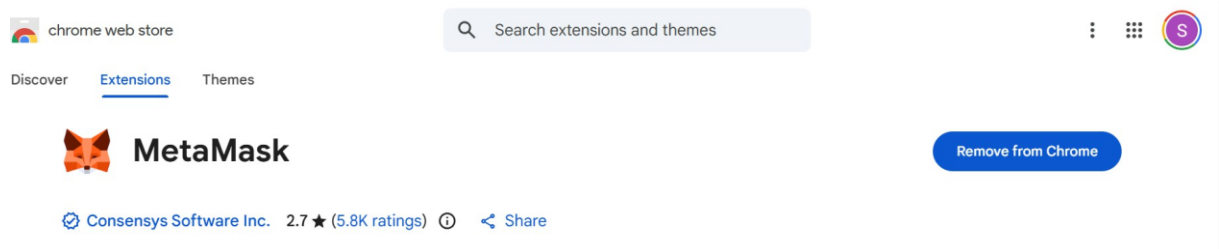
- Click Deploy in Remix
- Confirm transaction in MetaMask

Step 9: Perform Transactions

- Call contract functions
- Each transaction will require MetaMask confirmation

Implementation-

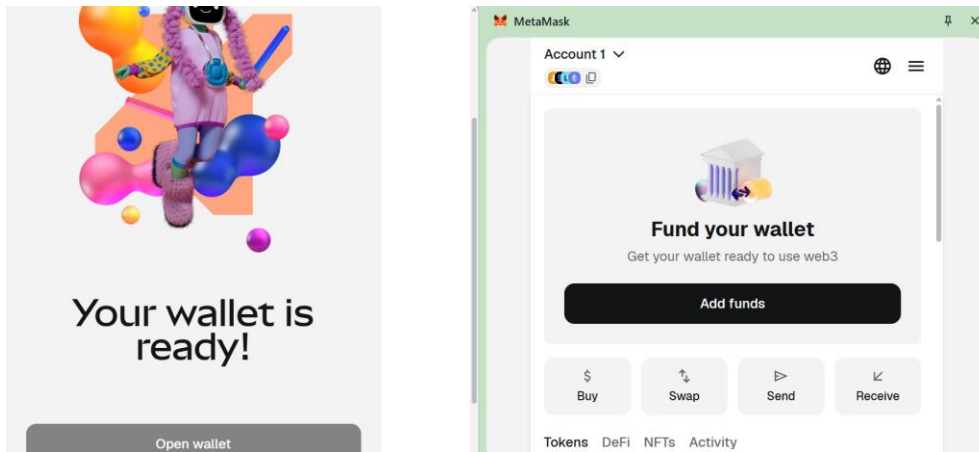
Install metamask



Create a new metamask wallet and give it a password

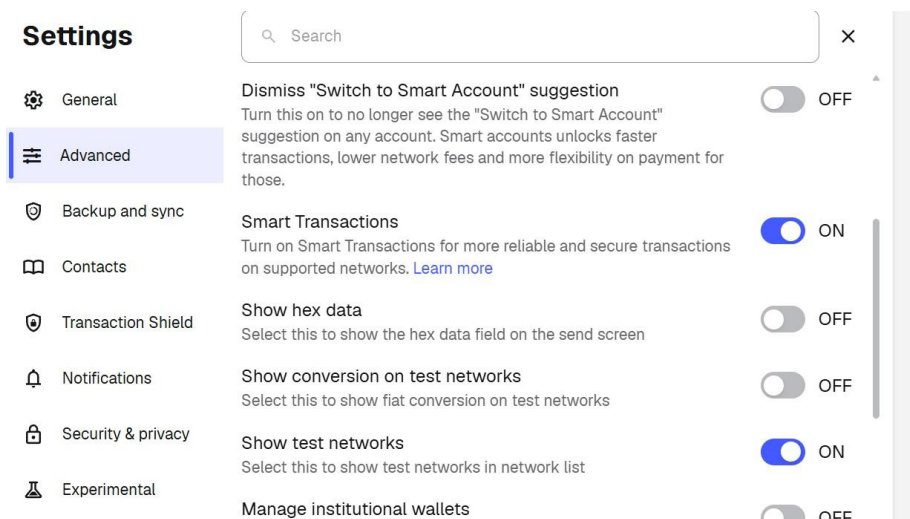
A screenshot of the MetaMask mobile app's password creation screen. It features a back arrow at the top left. The title 'MetaMask password' is prominently displayed. Below the title, a warning message states: 'Losing this password means losing wallet access on all devices, MetaMask can't reset it.' There are two input fields: 'Create new password' and 'Confirm password'. Each field has a toggle icon (an eye) to the right, likely for showing or hiding the password. A note below the first field says 'Must be at least 8 characters'.

Account is successfully created

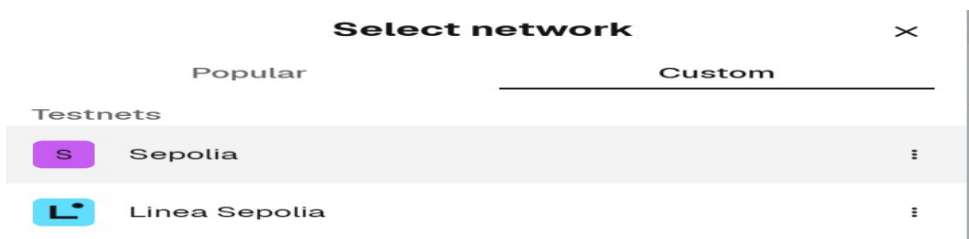


Steps to get funds from Sepolia Testnet

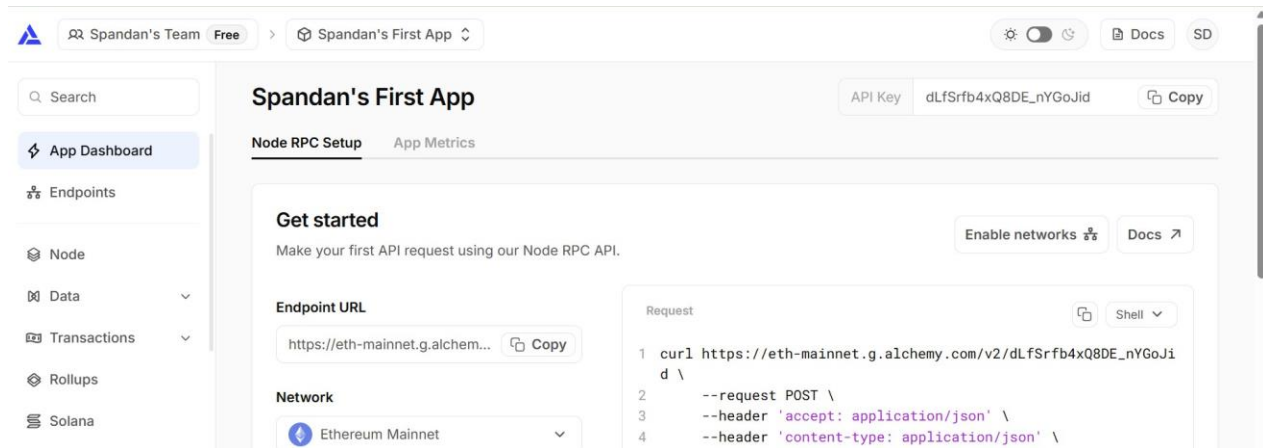
In settings switch on to show test networks



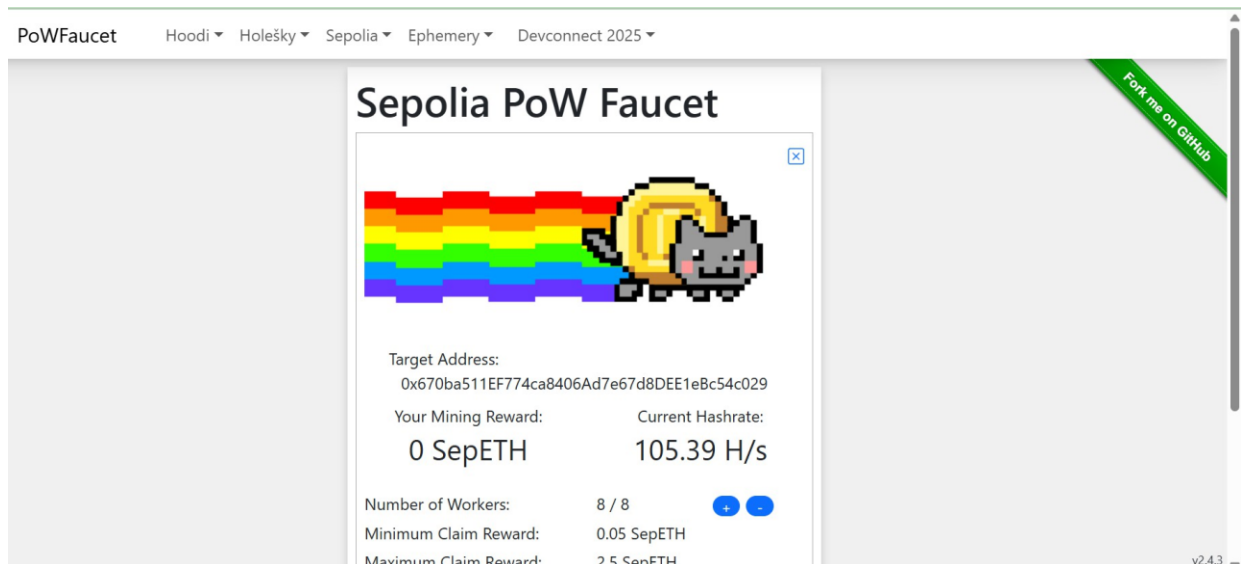
Select sepolia testnet



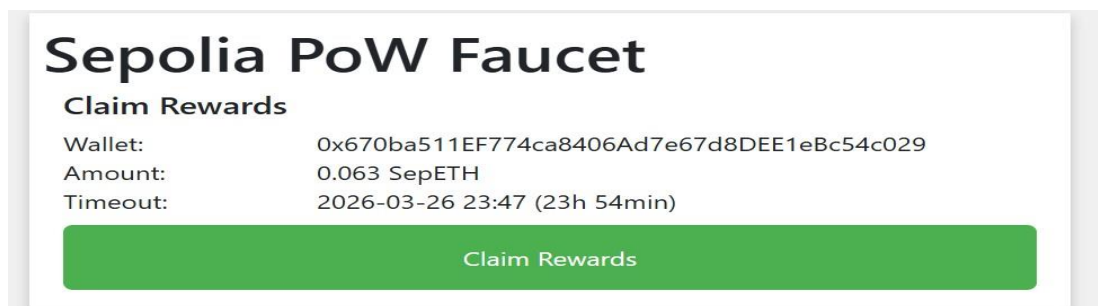
Create a free account in alchemy



Alchemy now requires this at least 0.001 ETH on Ethereum Mainnet, Shifting to PoW faucet.



Click on Claim rewards



Transaction is confirmed

Sepolia PoW Faucet

Claim Rewards

Wallet: 0x670ba511EF774ca8406Ad7e67d8DEE1eBc54c029
Amount: 0.063 SepETH
Timeout: -

Claim Transaction has been confirmed in block #10520426!

TX: [0x9552849a510aa5196ccdabc90780008599cb9a33341743959781682f84879baa](#)

Did you like the faucet? Give that project a  Star 5,456

Or support this faucet by sharing your result with a  Tweet  Post

[Return to startpage](#)

Account Balance



Account 1 



0.0626 SepoliaETH

+\$0.00 (+0.00%) [Discover](#) 

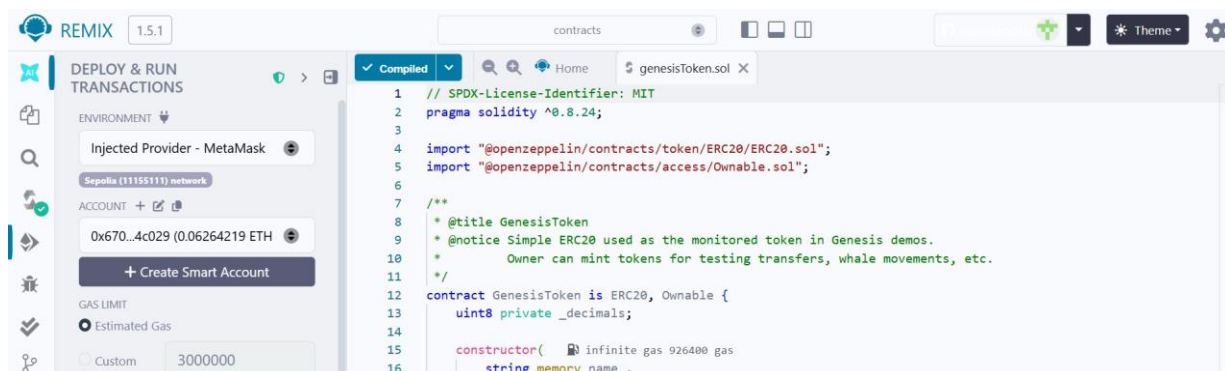
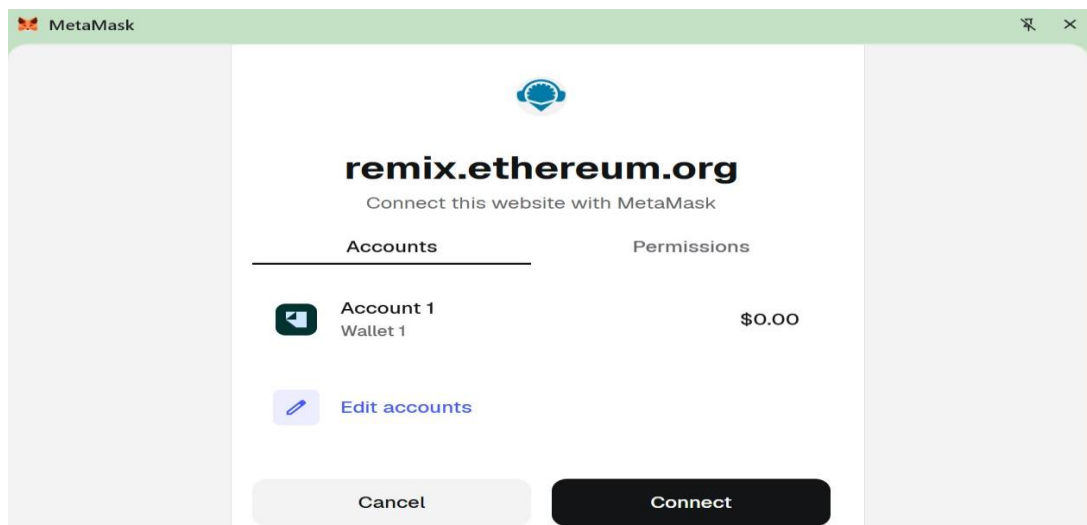
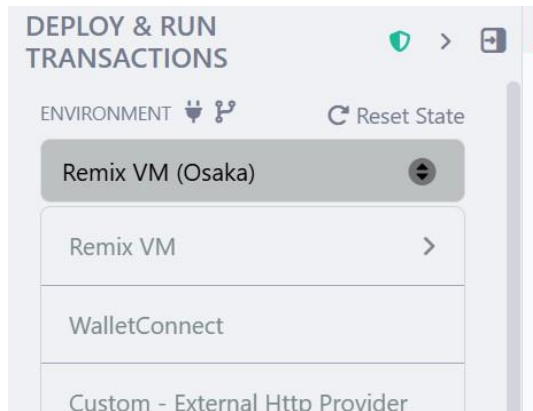
 Buy

 Swap

 Send

 Receive

Go to Remix IDE ,compile the smart contract in deploy under environment select wallet connect



GenesisToken.sol

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.24;
```

```
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
```

```
import "@openzeppelin/contracts/access/Ownable.sol";
```

```
/**
```

```
 * @title GenesisToken
```

```
 * @notice Simple ERC20 used as the monitored token in Genesis demos.
```

```
 *      Owner can mint tokens for testing transfers, whale movements, etc.
```

```
 */
```

```
contract GenesisToken is ERC20, Ownable {
```

```
    uint8 private _decimals;
```

```
    constructor(
```

```
        string memory name_,
```

```
        string memory
```

```
        symbol_, uint8
```

```
        decimals_, uint256
```

```
        initialSupply
```

```
    ) ERC20(name_, symbol_) Ownable(msg.sender) {
```

```
        _decimals = decimals_;
```

```
        _mint(msg.sender, initialSupply * (10 ** decimals_));
```

```
    }
```



```
function decimals() public view override returns (uint8) {  
  
    return _decimals;  
  
}
```

```
function mint(address to, uint256 amount) external onlyOwner {  
  
    _mint(to, amount);  
  
}
```

```
function burn(uint256 amount) external {  
  
    _burn(msg.sender, amount);  
  
}
```

Contract Deployed



Transaction Request

MetaMask

Account 1

ⓘ ⚙

Deploy a contract

This site wants you to deploy a contract

Estimated changes ⓘ

No changes

Network

s Sepolia

Request from ⓘ

remix.ethereum.org

Network fee ⓘ  0.0005

s SepoliaETH

Speed

Market ~12 sec

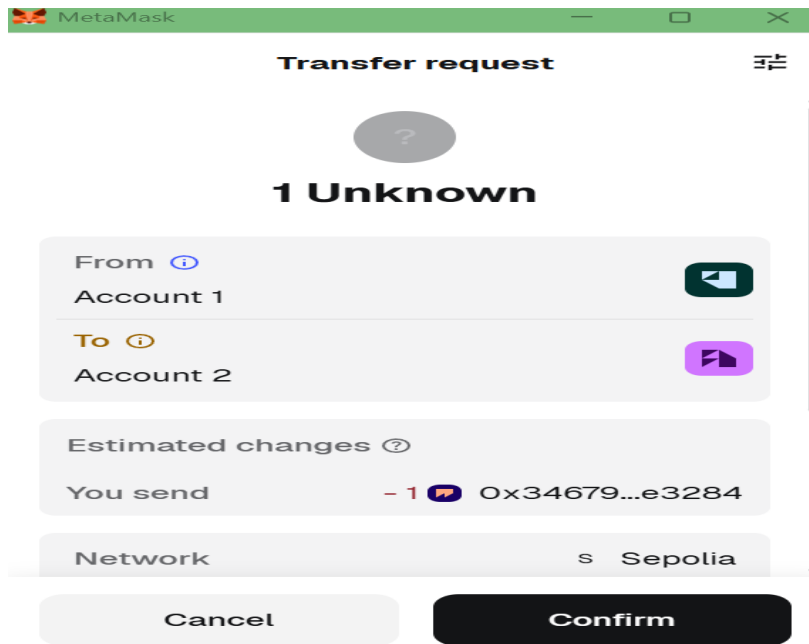
Cancel

Confirm

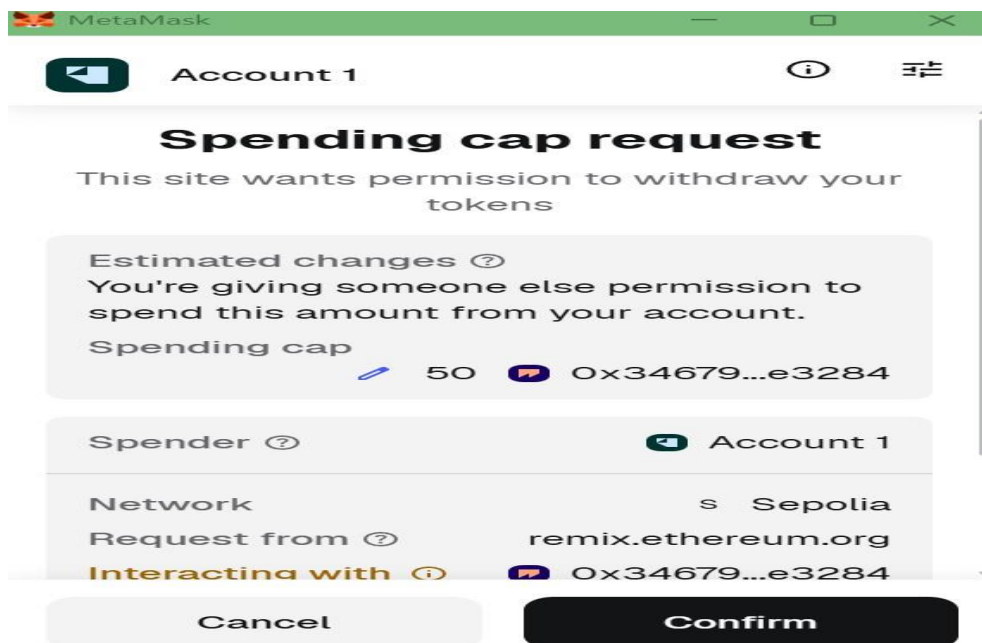
Confirmation on Metamask

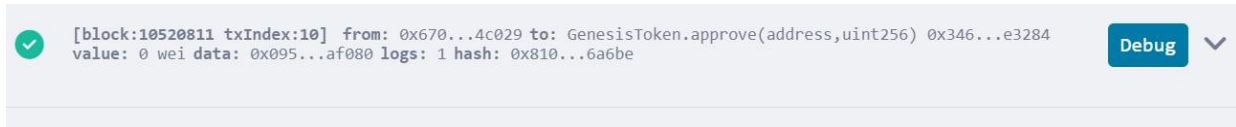
Contract deployment		×
Status		View on block explorer
Confirmed		Copy transaction ID
From		To
 0x670ba...4c029	 New contract	
Transaction		
Nonce		0
Amount		-0 SepoliaETH
Gas Limit (Units)		1230881
Gas Used (Units)		1220031
Base fee (GWEI)		0.0000000018
Priority fee (GWEI)		1.5
Total gas fee		0.00183 SepoliaETH

Transferred 1 token to another account via metamask



Approved the spending of tokens





Contract details on metamask

Tokens	DeFi	NFTs	<u>Activity</u>
Sepolia ▼			
Mar 26, 2026			
			
Approve GT spending cap			
Confirmed			
			
Sent Token			
Confirmed			
-0 SepoliaETH			
			
Contract deployment			
Confirmed			
-0 SepoliaETH			
-0 SepoliaETH			
			
Contract deployment			
Confirmed			
-0 SepoliaETH			
-0 SepoliaETH			

Conclusion-

The embedding wallet MetaMask was successfully integrated with Remix IDE to interact with smart contracts written in Solidity. Transactions such as deployment and token transfer were executed on the Sepolia Testnet. This experiment demonstrated secure wallet connectivity, transaction signing, and practical implementation of blockchain-based operations.